Moving off Firebase to a fully self-hosted local server (192.168.1.121)

Goal: zero paid or cloud dependency (no Firebase, no Vercel). The backend, database, and file storage would all run on your own office network box, `192.168.1.121`. No monthly subscription, and the app would keep working even without an internet connection.

This is a genuine rewrite, not a simple settings change. The backend currently talks to three separate Firebase services in roughly 20 files: Firestore (the database), Firebase Auth (handles logins and passwords), and Firebase Storage (holds employee documents). Every step below is required work, not optional polish.

---

0. What's actually being replaced

A search through `main/backend` for the code patterns that talk to Firebase turned up the real scope of the change:

FIREBASE PIECE	REPLACEMENT	WHY THIS CHOICE
Firestore (the database)	**MongoDB**, installed locally	Firestore stores records without a rigid, fixed shape (each entry can carry whatever fields it needs). MongoDB works the same way. Switching to a traditional table-based database (like Postgres) would mean designing formal tables and relationships for around 18 different record types from scratch. MongoDB lets each existing "look up this record" call become MongoDB's equivalent version with the surrounding logic left untouched. Less rewriting, less risk of breaking something.
Firebase Auth (creating accounts, looking people up, verifying passwords)	**bcrypt** (a well-established password-hashing tool) **plus your own `users` collection**	Firebase currently owns the scrambled (hashed) version of every password, in a format that can't be exported into bcrypt's format. This is the one part of the move that isn't a simple mechanical swap. See step 4.
Firebase Storage (where uploaded files are kept)	**A local disk folder**, served directly by the app	`multer`, the tool that already handles file uploads in this app, doesn't need a cloud storage service behind it. There's no need to add a separate storage system just for one feature (employee document uploads in `hrDeskController.js`).
Firestore's "transaction" feature (used in `sessions.js`, `approvalController.js`, `complaintControllerFactory.js` to make sure a group of related database changes either all happen or none do)	**MongoDB's own client sessions and transactions** feature	MongoDB's transactions require the database to be running in a mode called a "replica set" (even a single-machine one works), see step 1.
Firestore's "batch" feature (grouping several writes into one operation, used in `complaintControllerFactory.js` and `securityController.js`)	MongoDB's **`bulkWrite()`**	A direct equivalent, does the same job.

The record types (called "collections") found in use, which all need to exist in MongoDB too: `users`, `sessions`, `failed_logins`, `audit_logs`, `settings` (which itself holds several named configuration entries: notification rules, action permissions, role permissions, system config, SLA policies), `hr_complaints`, `it_complaints`, `approvals`, `leave_requests`, `assets`, `departments`, `employees`, `attendance`, `extra_hours`, `sent_emails`, `tasks`, `projects`, `renders`.

---

1. Install MongoDB on 192.168.1.121

1. Download **MongoDB Community Server** (the Windows installer) from mongodb.com. It's free and doesn't require an account.
2. Install it as a Windows background service (the installer does this by default), so it starts automatically whenever the server restarts.
3. **Turn on "replica set" mode**, which is required for the transaction feature mentioned above; without it, those parts of the app would fail when they run.
   - Open `C:\Program Files\MongoDB\Server\<version>\bin\mongod.cfg` and add:

     ```
     replication:
       replSetName: "rs0"
     ```

   - Restart the MongoDB service (open `services.msc`, find MongoDB, click Restart).
   - Then, just once, open MongoDB's command-line tool (`mongosh`) and run:

     ```
     rs.initiate()
     ```

4. Confirm it's working: running `mongosh mongodb://localhost:27017` should connect successfully.
5. By default it only listens on the machine's own internal address (`127.0.0.1:27017`), which is fine, since the backend program runs on the very same machine and talks to it locally, not over the office network.

---

2. Swap the Firebase setup file for a MongoDB one

Replace `main/backend/config/firebase.js` with a new file, `main/backend/config/db.js`:

```
const { MongoClient } = require('mongodb');
require('dotenv').config();

const client = new MongoClient(process.env.MONGO_URI || 'mongodb://localhost:27017/fu
let db;

async function connect() {
  if (db) return db;
  await client.connect();
  db = client.db();
  return db;
}

module.exports = { client, connect, getDb: () => db };
```

Add `mongodb` as a dependency (run `npm install mongodb`), and remove `firebase-admin`.

`server.js` needs to wait for `connect()` to finish before it starts accepting requests (`await connect()` before `app.listen(...)`). Firestore used to connect automatically the first time it was needed; MongoDB's connection has to be started explicitly.

---

3. Rewrite every place the code talks to the database

This is the bulk of the work: 19 controllers (the files that handle each feature), plus `utils/sessions.js`, `utils/auditLog.js`, `utils/notificationRules.js`, `middleware/authMiddleware.js`, and `middleware/permissionMiddleware.js`. The translation from Firestore's way of doing things to MongoDB's is mechanical, meaning each change follows a repeatable pattern, but it still has to be done file by file, carefully.

FIRESTORE'S WAY	MONGODB'S EQUIVALENT
Look up one record by ID	Find one record matching that ID; if nothing matches, you get back nothing instead of an empty placeholder
Save a record, replacing whatever was there	Replace it, creating it if it didn't already exist
Update just some fields on a record, leaving the rest alone	Update just those fields
Update just some fields (a second, equivalent way Firestore offers)	Same update operation as above
Delete a record	Delete a record
Create a new record	Create a new record; note that you need to capture the new record's ID afterward, since MongoDB doesn't hand you back a simple text ID the way Firestore does. You can either let MongoDB generate its own ID format or set your own.
Find every record matching a condition	Find every record matching a condition
Find matching records, sorted and limited to a certain number	Find, sort, and limit, same idea
Count matching records	Count matching records
A Firestore record's ID (a plain text value, for example `"AST-1006"`)	Use that exact same value as MongoDB's ID field, this keeps the existing convention of "the business ID is also the database ID" working without any other code changes

Work through this **one file at a time**, and test that file's features by hand (using a tool like Postman or the `curl` command) before moving to the next one. Do them in this order, riskiest and most foundational first: `authController.js` and `middleware/authMiddleware.js` first (nothing else works if login is broken), then `complaintControllerFactory.js` (used by both the HR and IT complaint features, and the biggest file), then everything else.

Grouped changes (transactions and batches) specifically

- `utils/sessions.js` (the part that issues new login tokens), `approvalController.js`, and `complaintControllerFactory.js` (both in their status-update code) use Firestore's transaction feature, which guarantees a group of changes either all succeed together or none of them happen. MongoDB's equivalent:

  ```
  const session = client.startSession();
  try {
    await session.withTransaction(async () => {
      // reads/writes here, passing { session } as the last argument to each one
    });
  } finally {
    await session.endSession();
  }
  ```

- `complaintControllerFactory.js` (its delete-complaint code) and `securityController.js` use Firestore's batch feature, for grouping several writes into one. MongoDB's equivalent: build a list of the delete/update operations you want, and hand the whole list to MongoDB's `bulkWrite()` function at once.

---

4. Replace Firebase Auth with your own local password login

This is the security-sensitive part. Don't rush or shortcut it.

The scrambled passwords currently stored by Firebase Auth cannot be exported into bcrypt's format. Firebase uses a different scrambling method with settings unique to your Firebase project, and there's no supported way to check those existing scrambled passwords outside of Firebase itself. You have two options:

- **(Recommended for a project this size)** Require every existing user to reset their password at the moment of the switchover. Either email each person a one-time reset link, or have an admin set a temporary password for them through the existing "force reset" feature in `superAdminUserController.js`, once that feature itself has been rewritten (see below). This is simple, leaves no room for confusion, and is a perfectly reasonable ask for an internal company tool.
- A more advanced alternative: implement Google's own published version of their password-scrambling algorithm, run against an exported dump of the old Firebase password data, so people's existing passwords would keep working without anyone resetting anything. This is considerably more work, and really only worth doing if asking everyone to reset their password isn't acceptable.

Assuming you go with the password-reset approach, here's what changes:

- **`config/firebase.js`'s login-handling export**: delete it entirely. Password checking moves into the `users` collection itself.
- **`authController.js`**:

    - The sign-up code: instead of creating the account through Firebase, scramble the password with bcrypt and store that scrambled version directly on the new user's record (along with their email, name, role, and so on), generating your own unique ID for them instead of relying on a Firebase-issued one.
    - The login code: instead of checking the password through Firebase's online verification service, compare it locally using bcrypt's built-in comparison function.
    - Delete the leftover Firebase-specific configuration values and the emulator-related code entirely, since none of it applies anymore.
- **`superAdminUserController.js`**: every place it currently calls Firebase to create a user, disable/enable a user, delete a user, or reset someone's password needs to be replaced with the equivalent direct database operation: inserting a new record with a bcrypt-scrambled password, flipping an "active" true/false field that already exists in the data (the login check in `authController.js` already relies on it), deleting the record outright, or updating the record with a freshly bcrypt-scrambled new password.
- Add `bcrypt` as a project dependency (`npm install bcrypt`).
- Remove the Firebase API key setting from your configuration; it's no longer needed once password verification switches over to bcrypt entirely.

The login-token system (JWTs, cookies, and the ability to remotely sign a session out) doesn't need to change at all. None of that is specific to Firebase; it stays exactly as it is today.

---

5. Replace Firebase Storage with local disk storage

`hrDeskController.js`'s employee-document upload feature is the only place that touches Firebase Storage. Replace it with something like:

```
// instead of uploading to Firebase Storage and generating a signed web link
const fs = require('fs/promises');
const path = require('path');

const UPLOAD_ROOT = path.join(__dirname, '..', 'uploads', 'employee-documents');
await fs.mkdir(path.join(UPLOAD_ROOT, id), { recursive: true });
const diskPath = path.join(UPLOAD_ROOT, id, `${docType}-${Date.now()}-${safeName}`);
await fs.writeFile(diskPath, req.file.buffer);
const url = `/uploads/employee-documents/${id}/${path.basename(diskPath)}`;
```

Then, in `server.js`, make that folder directly downloadable from the app itself:

```
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
```

Make sure `main/backend/uploads/` is excluded from the project's shared code history (the actual uploaded files shouldn't ever be committed alongside the code). Employee documents contain personal information. This folder is only reachable from your office network, same as everything else described here, but double-check the server itself is never exposed directly to the open internet.

6. Configuration values

The new `main/backend/.env` file would look like:

```
MONGO_URI=mongodb://localhost:27017/fute_portal
JWT_SECRET=<keep existing value>
SMTP_HOST=... SMTP_PORT=... SMTP_USER=... SMTP_PASS=...   # unchanged
HR_EMAIL=... IT_EMAIL=...   # unchanged
PORT=5000
```

Remove every Firebase-related setting; nothing reads them anymore once step 4 above is done.

---

7. Run the backend on the server as a permanent background service

Copy the `main/backend` folder to the server, then run:

```
cd main/backend
npm install
npm install mongodb bcrypt
npm uninstall firebase-admin
```

Use a tool called **PM2** to keep it running permanently, restarting it automatically if it crashes or the machine reboots (this is the standard, free choice for this; there's no need to build a custom Windows service from scratch):

```
npm install -g pm2
pm2 start server.js --name fute-backend
pm2 save
pm2-installer   # or the equivalent "pm2 startup" command, so it restarts automatical
```

The backend would then be reachable at `192.168.1.121:5000`.

---

8. Build and serve the frontend (the part people actually see and use)

```
cd main/frontend
npm install
npm run build
```

Then serve the resulting `dist` folder from the same machine. The simplest way is a small tool called `serve`:

```
npm install -g serve
serve -s dist -l 80
```

(Or, if you'd rather use Windows' built-in web server since the machine already runs Windows, point IIS at the `dist` folder instead.)

Before building, set `main/frontend/.env`'s web address setting to point at `http://192.168.1.121:5000`, and rebuild any time that value changes; the build tool (Vite) locks environment settings into the build permanently at build time, they can't be changed afterward without rebuilding.

---

9. Backups: Firestore handled this automatically, a single MongoDB machine does not

This is not optional for a system holding real employee data. Set up a daily automatic backup:

1. Create a `C:\backups\` folder.
2. In Windows Task Scheduler, set up a daily task that runs:

```
mongodump --uri="mongodb://localhost:27017/fute_portal" --out="C:\backups\%date:~
```

3. **Copy those backup files off the machine regularly** (to an external drive, or another machine on the network). A backup stored on the very same disk as the live data won't help if that disk fails.
4. To restore from a backup later: `mongorestore --uri="mongodb://localhost:27017/fute_portal" C:\backups\<date>\fute_portal`

---

10. Network access

- The server should only be reachable from `192.168.1.0/24` (your office network) unless you deliberately open it to the wider internet, which isn't recommended unless you also add encrypted connections (TLS) and tighten the list of allowed website addresses in `server.js` beyond just your local development address.
- Windows Firewall needs to allow incoming connections on port 5000 (the backend) and port 80 (the website) so other computers on the office network can reach it. Restrict that to your office network's address range specifically, not "any" source.

---

11. Cutover checklist

- [ ] MongoDB installed, running in replica-set mode, reachable through `mongosh`
- [ ] `config/db.js` replaces `config/firebase.js`
- [ ] All 19 controllers plus the 4 supporting files converted and individually tested

- [ ] `authController.js` and `superAdminUserController.js` fully switched to bcrypt, with no remaining Firebase login calls anywhere (searching the code for Firebase imports turns up nothing)
- [ ] Existing users notified and their passwords reset before old logins stop working
- [ ] `hrDeskController.js` uploads go to local disk, and that folder is served directly by the app
- [ ] The `.env` configuration file has no leftover Firebase settings
- [ ] The backend runs under PM2 and restarts automatically after a reboot
- [ ] The frontend is built with its web address setting pointed at the server, and is being served
- [ ] A daily backup is scheduled, has been tested to confirm it actually restores, and copies are kept off the machine
- [ ] The firewall restricts both the backend and frontend ports to the office network only